

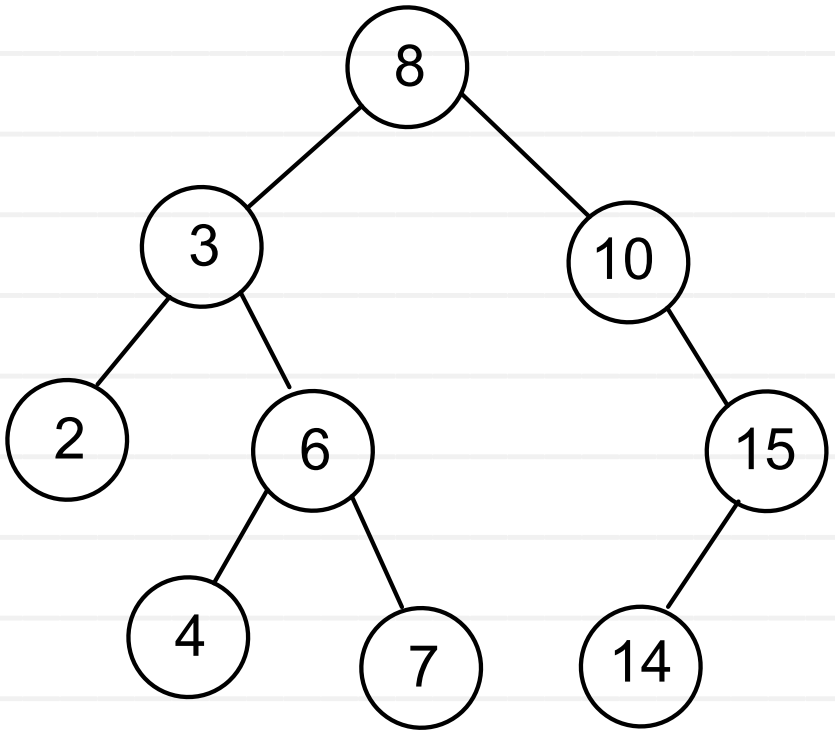
Sunbeam Institute of Information Technology
Pune and Karad

Algorithms and Data structures

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

Binary Search Tree - PreOrder Non recursive

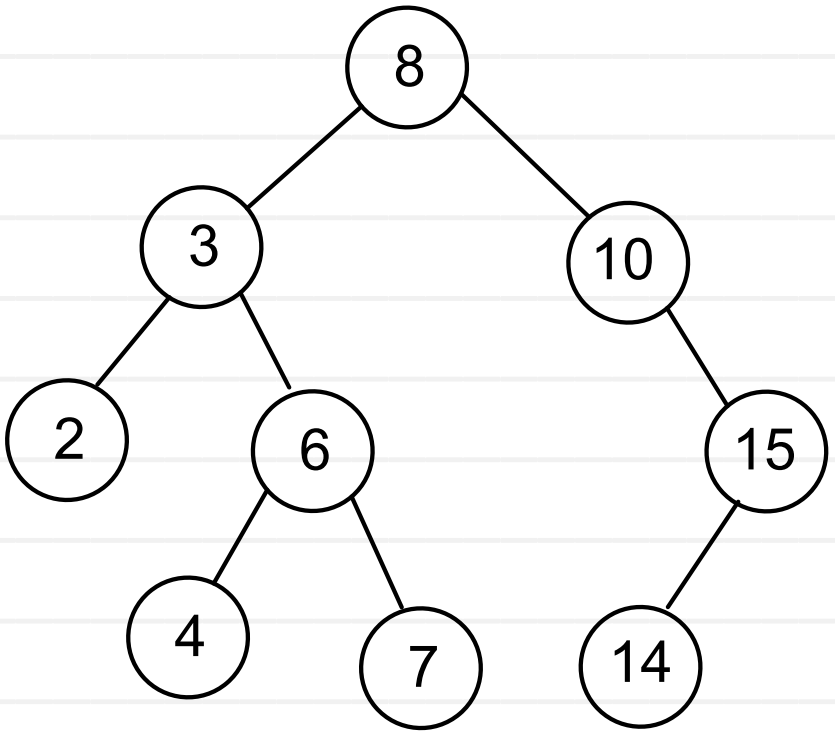


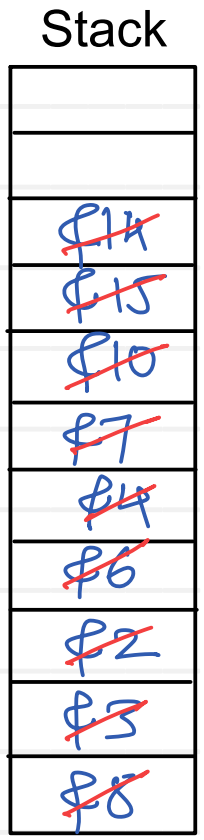


1. start from root node
2. traverse till extreme left
   - 2.1 visit node
   - 2.2 if right exist, push on stack
   - 2.3 go on left node
3. pop node from stack
4. repeat step 2 & 3 till all nodes are not visited.

Traversal : 8, 3, 2, 6, 4, 7, 10, 15, 14

Binary Search Tree - InOrder Non recursive

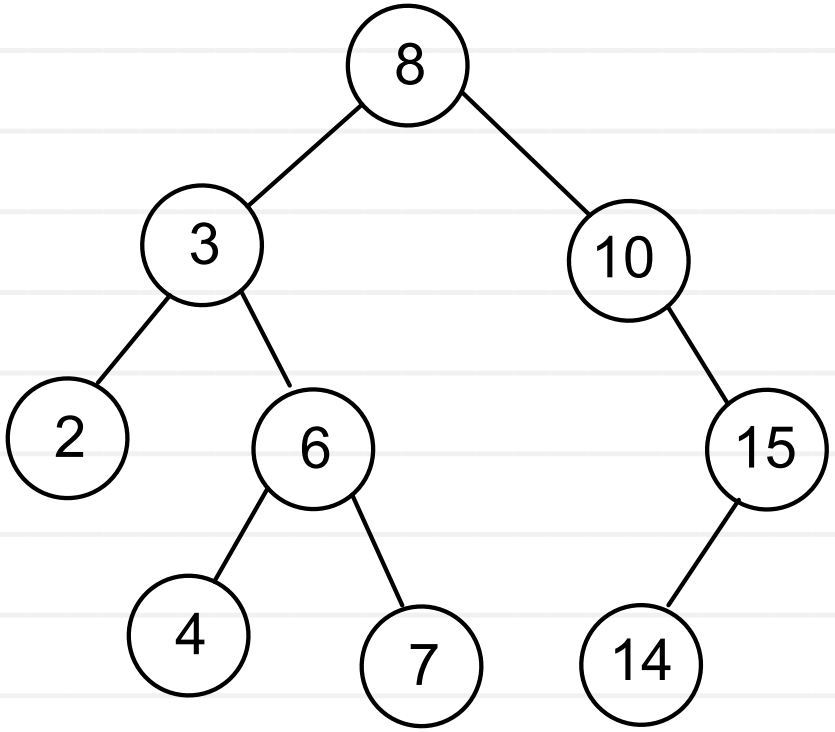




1. Start from root
2. traverse till extreme left
   2.1 push current node on stack
   2.2 go on left node
3. pop node from stack
4. visit current node
5. go on right of current node
6. repeat step 2 to 5 till all nodes are not visited

Traversal : 2, 3, 4, 6, 7, 8, 10, 14, 15

Binary Search Tree - PostOrder Non recursive

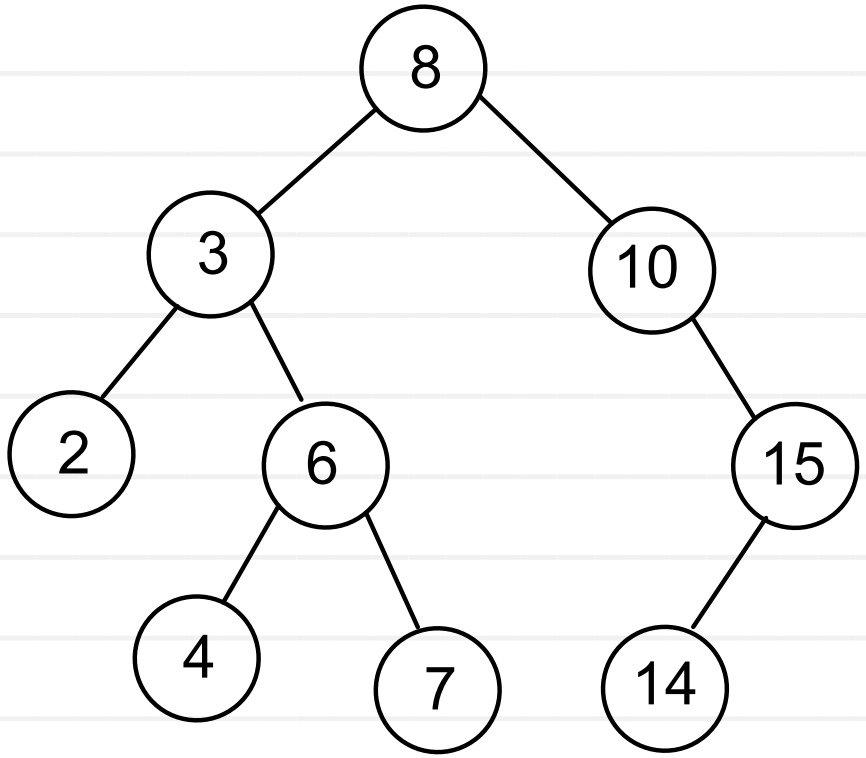


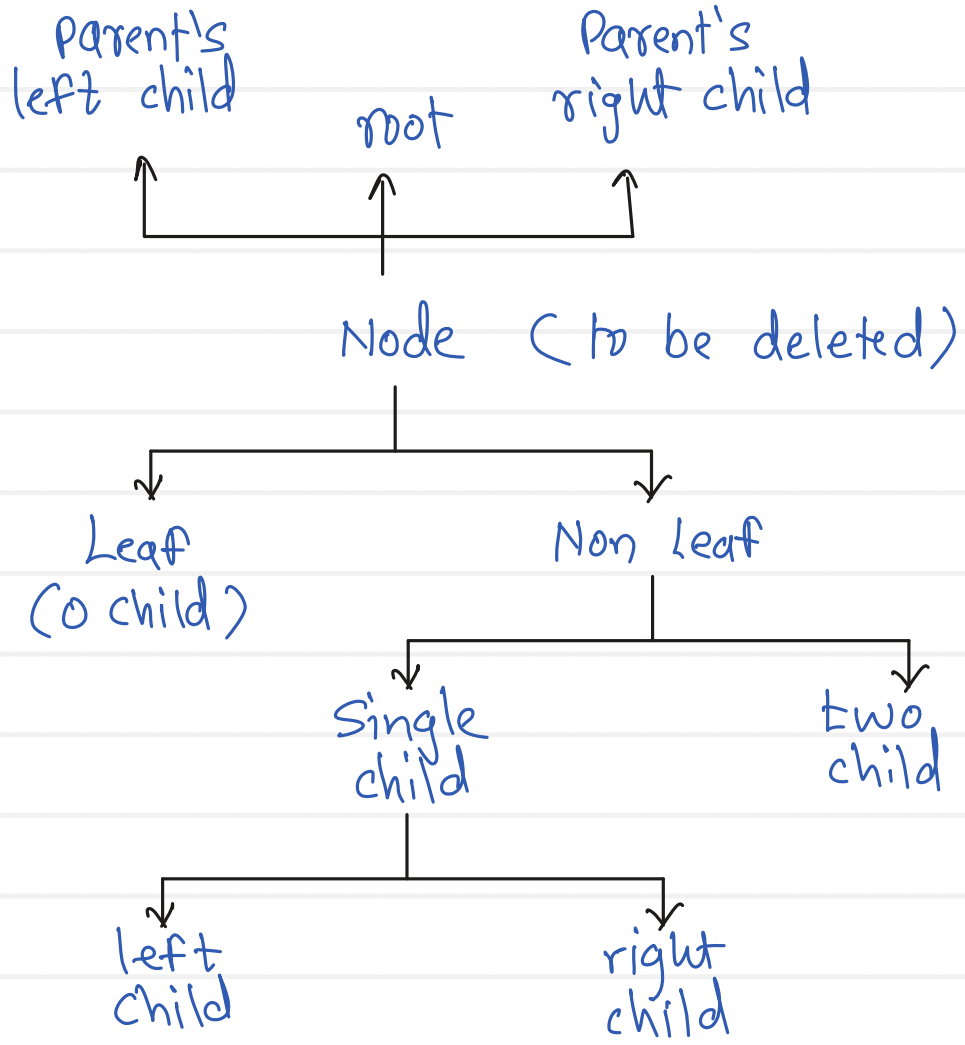


Traversal : 2, 4, 7, 6, 3, 14, 15, 10, 8

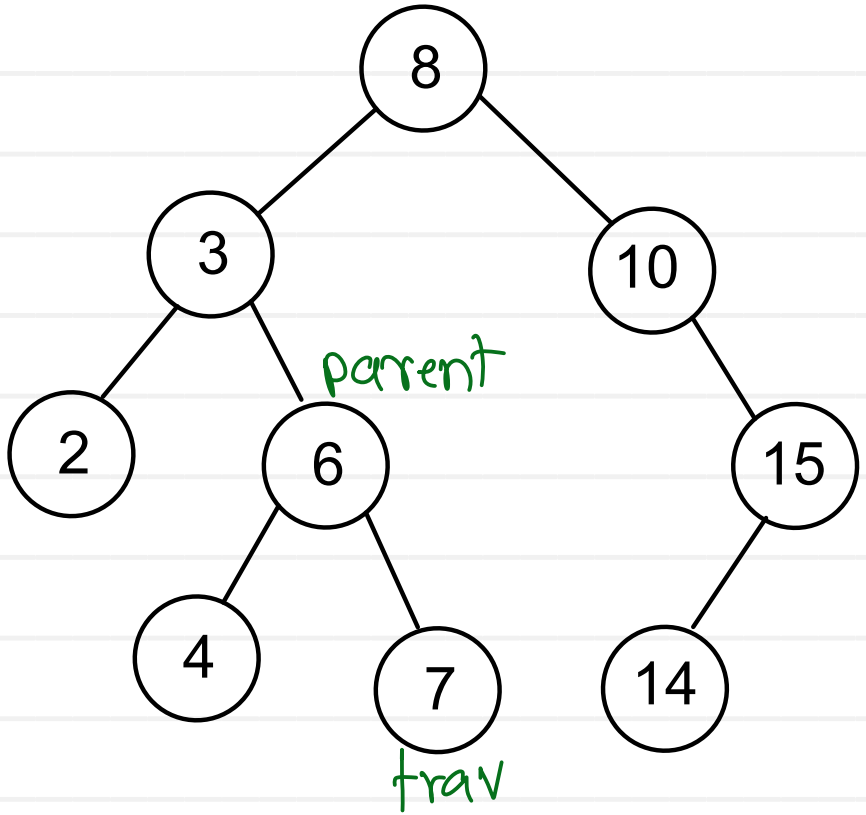
1. start from root node
2. traverse till extreme left
   2.1 push current node on stack
   2.2. go on left of current node
3. pop node from stack
4. if right doesn't exist or right is visited
   4.1 visit current node,
   4.2 mark node as visited
5. if right exist & not visited,
   push current node on stack
6. go on right node
7. repeat step 2 to 6 untill all node are visited.

Binary Search Tree - Delete Node





Binary Search Tree - Binary Search with Parent



key = 16

trav	parent
&8	null
&10	&8
&15	&10
null	&15

key = 7

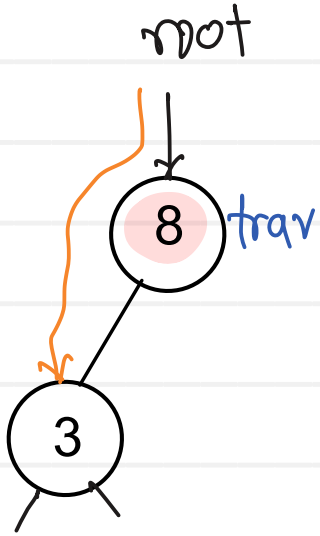
trav	parent
&8	null
&3	&8
&6	&3
&7	&6

key = 8

trav	parent
&8	null

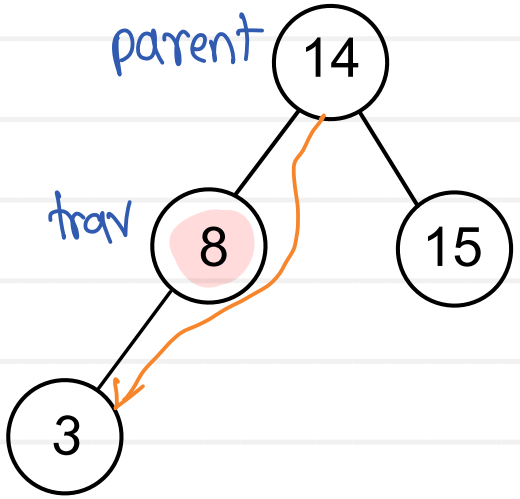
BST- Delete Node with Single child node (Left child)

a. root Node



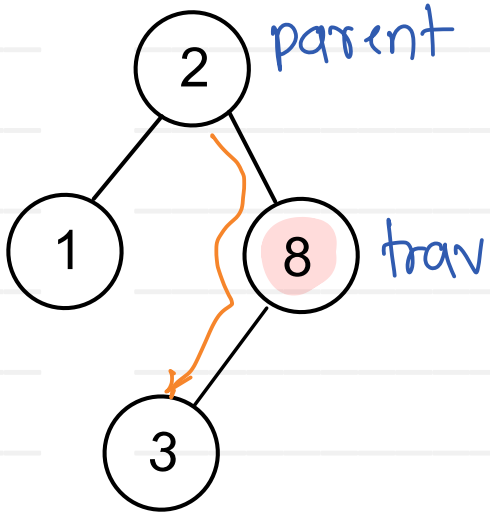
root = trav.left

b. Parent's left child



parent.left = trav.left

c. Parent's right child

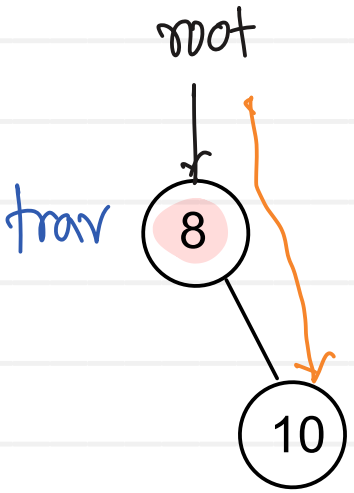


parent.right = trav.left

- connect left child with either root, parent's left or parent's right

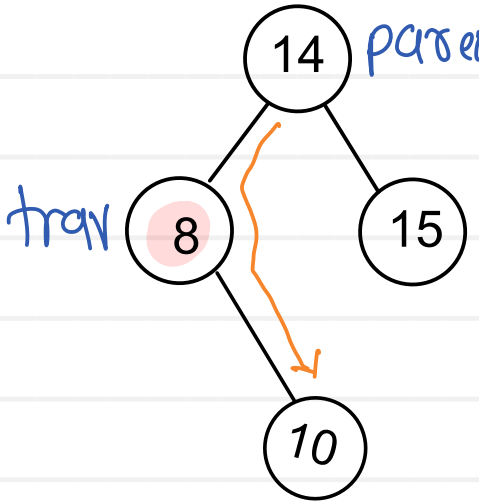
BST - Delete Node with Single child node (Right child)

a. root Node



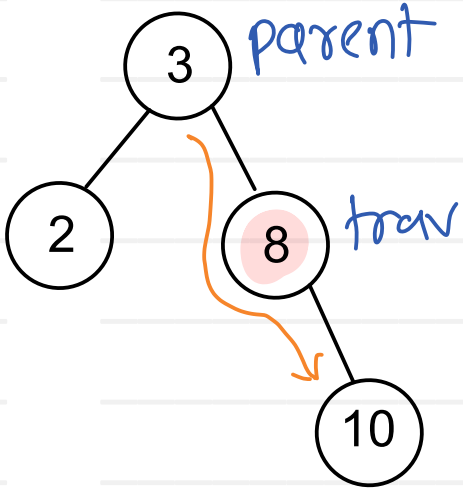
root = trav.right

b. Parent's left child



parent.left = trav.right

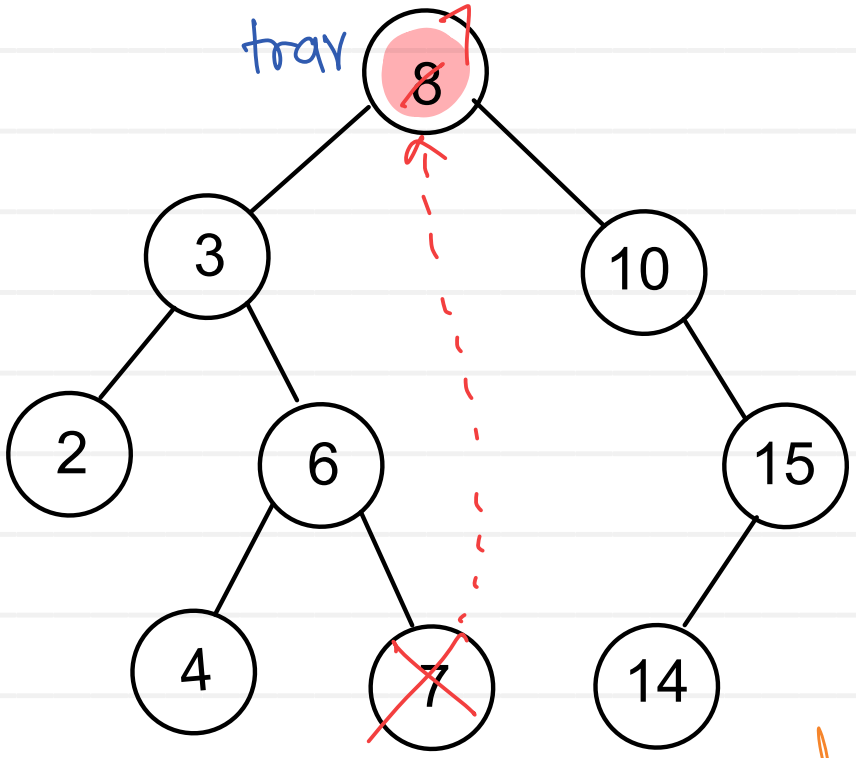
c. Parent's right child



parent.right = trav.right

- connect right child with either root, parent's left or parent's right

BST - Delete Node with Two child node



Inorder : 2 3 4 6 7 8 10 14 15

inorder predecessor → extreme right of left subtree

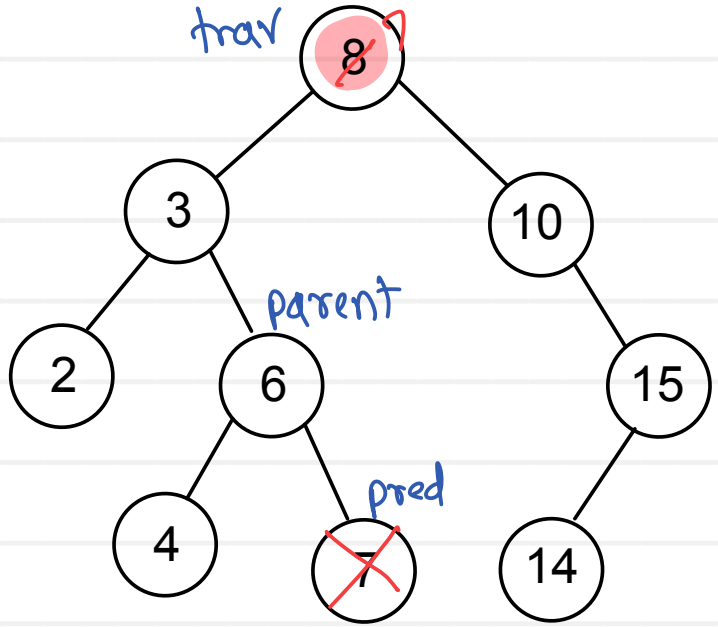
inorder successor → extreme left of right subtree

```
Node pred = trav.left;
parent = trav
while (pred.right != null) {
    parent = pred;
    pred = pred.right;
}
```

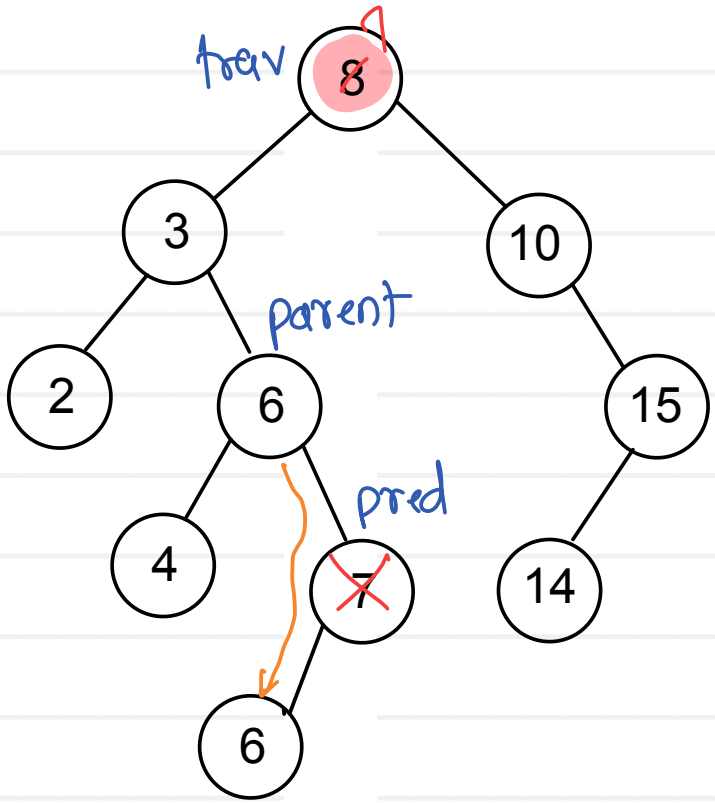
pred	parent
&3	&8
&6	&3
&7	&6

1. check for two child
2. find predecessor of a node
3. replace value of a node by predecessor value
4. delete predecessor

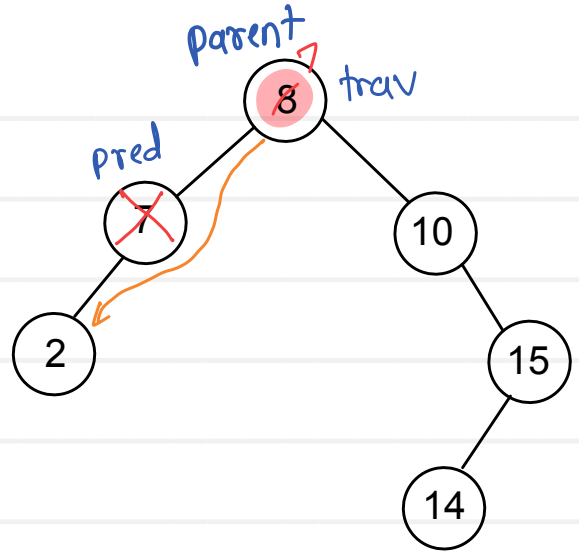
BST - Delete Node with Two child node

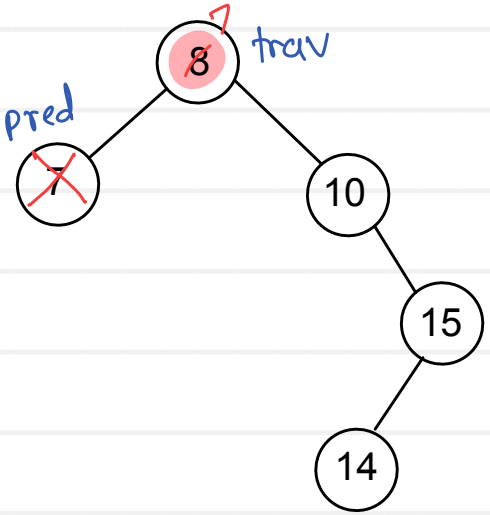


parent.right = null
or
parent.right = pred.left



parent.right = pred.left

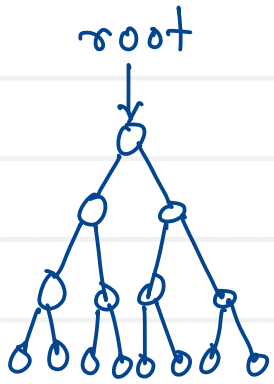




BST - Time complexity of operations

Capacity : max number of nodes for given height.

height (h)	No. of Nodes (n)
-1	0
0	1
1	3
2	7
3	15



$$n = 2^{h+1} - 1$$

$$2^{h+1} = n + 1$$

$$2^{h} \approx n$$

$$\log 2^{h} = \log n$$

$$h = \frac{\log n}{\log 2}$$

Time $\propto$ height

Time $\propto \frac{1}{\log 2} \log n$

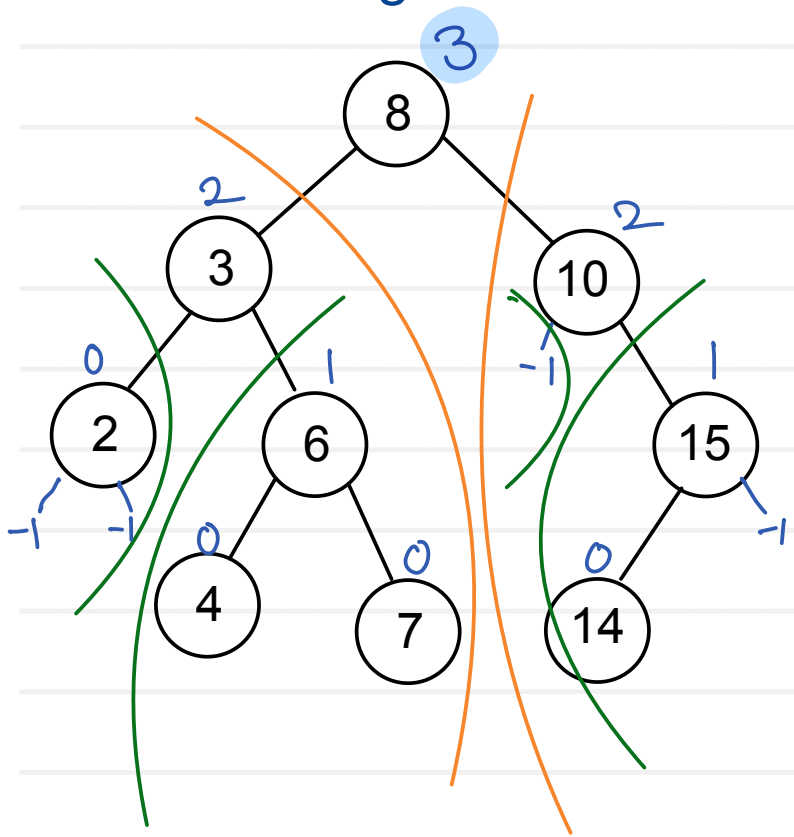
$$T(n) = O(\log n)$$

Add
Delete
search

Traverse – $T(n) = O(n)$

Binary Search Tree - Height

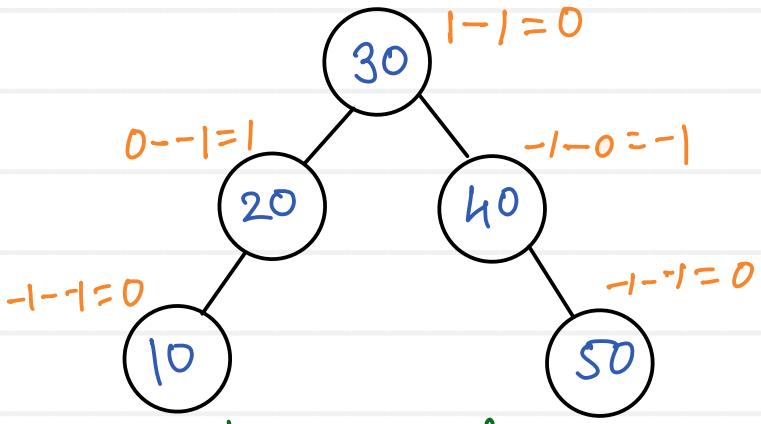
Height of root = MAX (height (left sub tree), height (right sub tree)) + 1



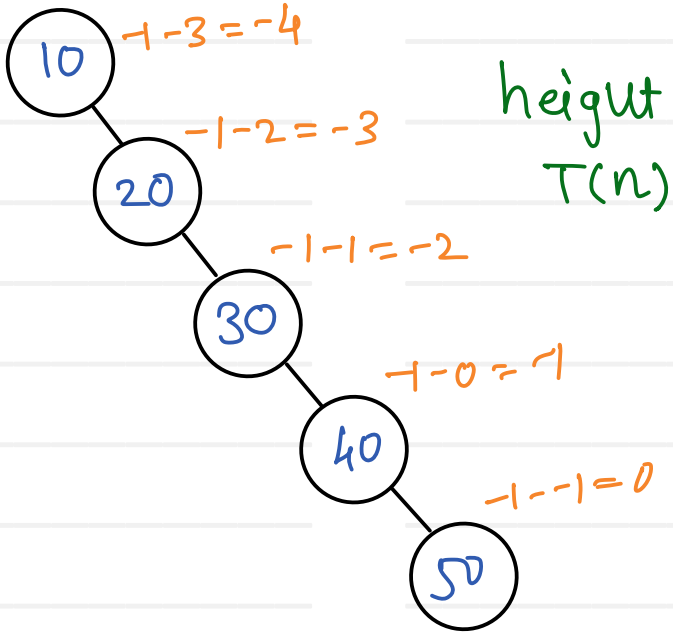
1. If left or right sub tree is absent then return -1
2. Find height of left sub tree
3. Find height of right sub tree
4. Find max height
5. Add one to max height and return

Skewed Binary Search Tree

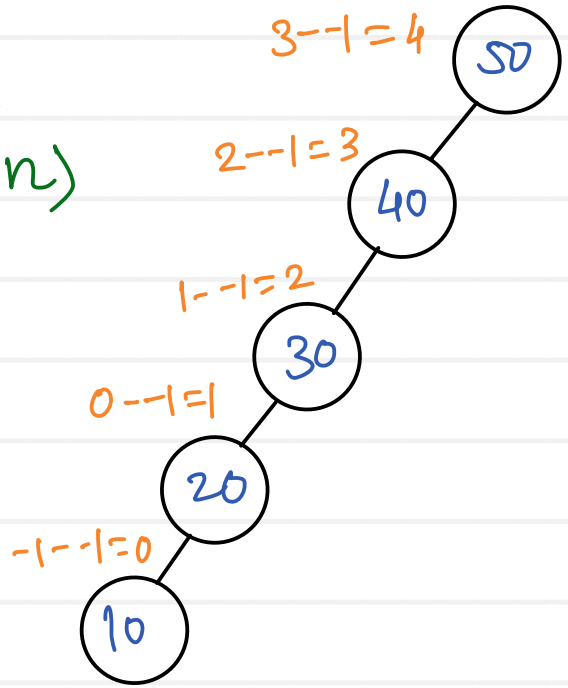
Keys : 30, 40, 20, 50, 10



Keys : 10, 20, 30, 40, 50



Keys : 50, 40, 30, 20, 10



- In binary tree if only left or right links are used, tree grows only in one direction such tree is called as skewed binary search tree
  - Left skewed binary search tree
  - Right skewed binary search tree
- Time complexity of any BST is O(h)
- Skewed BST have maximum height ie same as number of elements.
- Time complexity of searching is skewed BST is O(n)

Balanced BST

- To speed up searching, height of BST should be minimum as possible
- If nodes in BST are arranged, so that its height is kept as less as possible, is called as Balanced BST

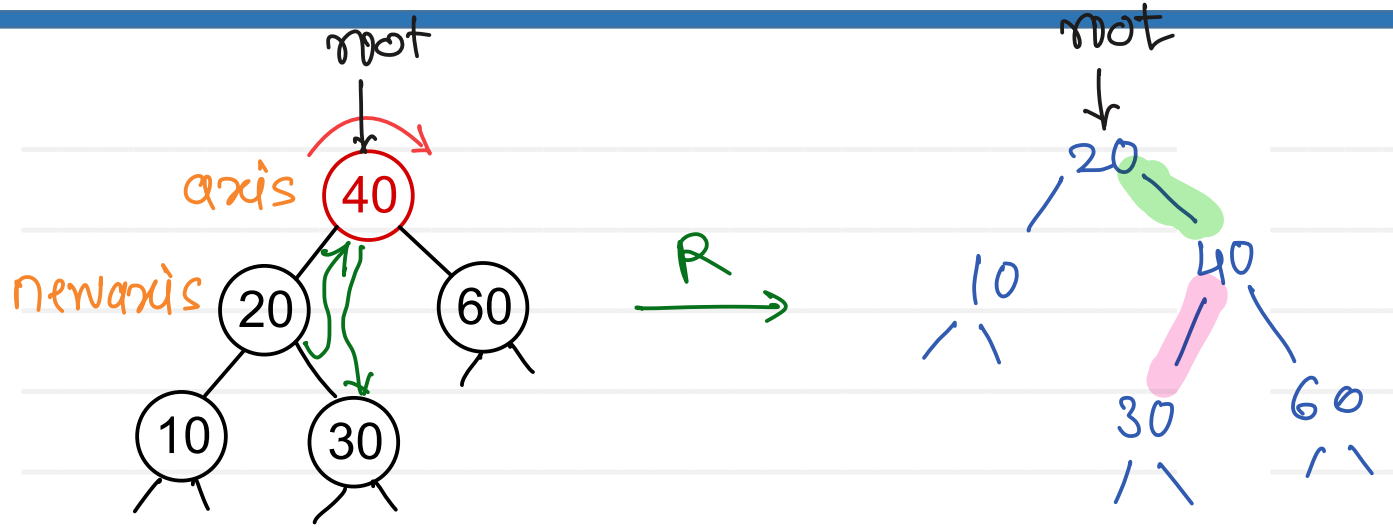
Balance factor (Node)	=	Height (left sub tree)	-	Height (right sub tree)

- tree is balanced if balance factors of all the nodes is either -1, 0 or +1
- balance factors = {-1, 0, +1}
- A tree can be balanced by applying series of left or right rotations on imbalance nodes (node which balance factor other than -1, 0 or +1)

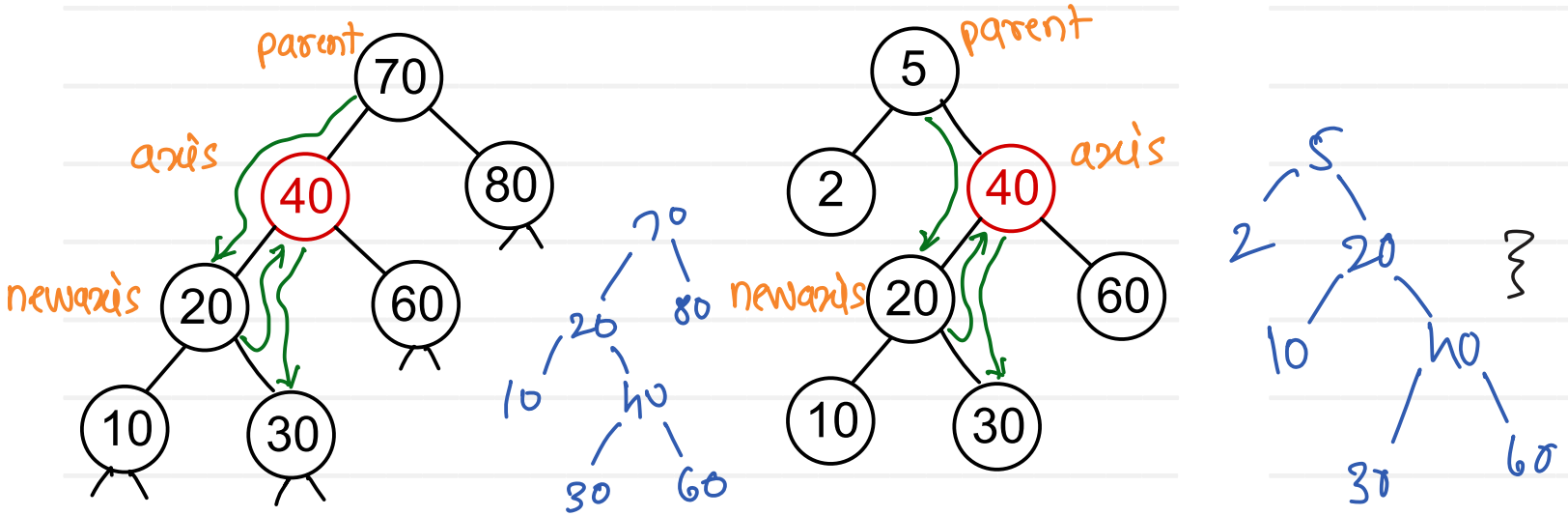
- types of balanced BST:
  1. AVL tree
  2. Red Black tree
  3. Splay tree

Right Rotation

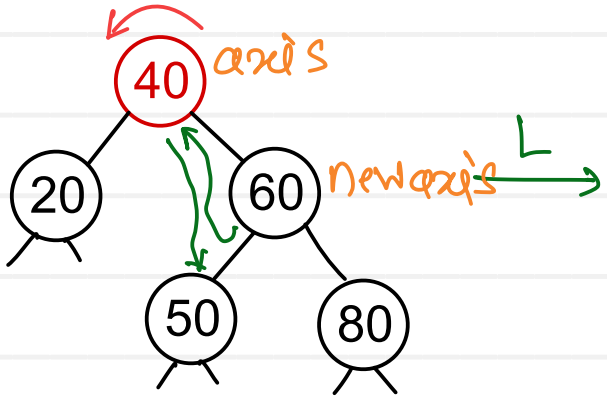
(let us assume 40 is imbalance)

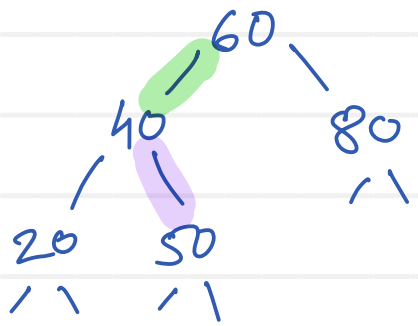



```
void rightRotation(parent, axis){
    newaxis = axis.left;
    axis.left = newaxis.right;
    newaxis.right = axis;
    if(axis == root)
        root = newaxis;
    else if(axis == parent.left)
        parent.left = newaxis;
    else if(axis == parent.right)
        parent.right = newaxis;
}
```

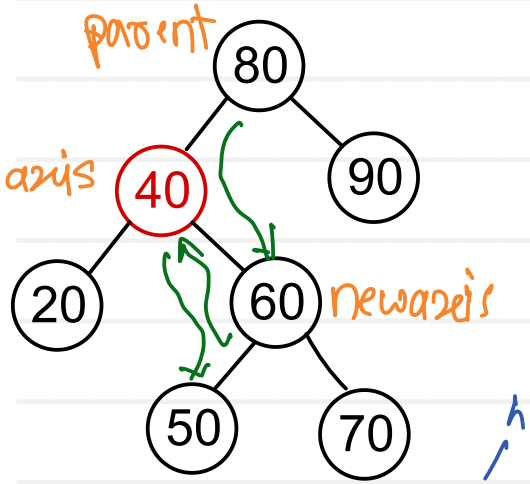


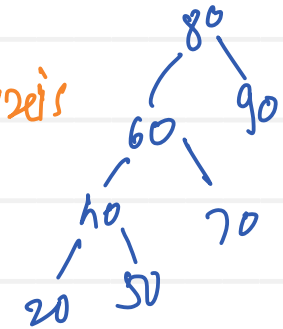
Left Rotation

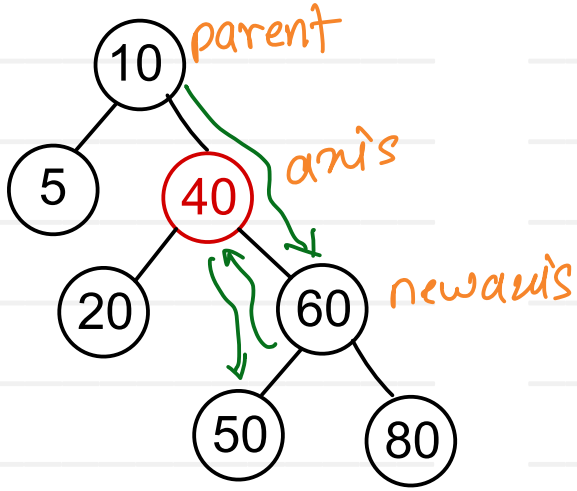


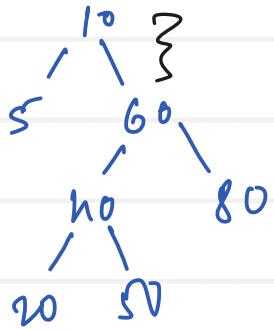



```
void leftRotation(parent, axis) {
    newaxis = axis.right;
    axis.right = newaxis.left;
    newaxis.left = axis;
    if(axis == root)
        root = newaxis;
    else if(axis == parent.left)
        parent.left = newaxis;
    else if (axis == parent.right)
        parent.right = newaxis;
```



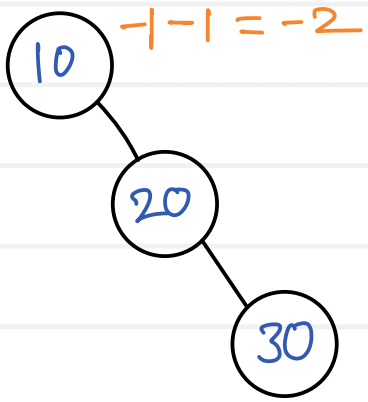






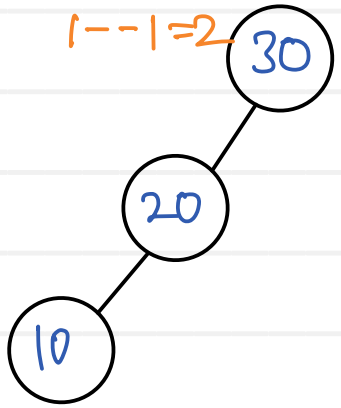
RR Imbalance

Keys : 10, 20, 30



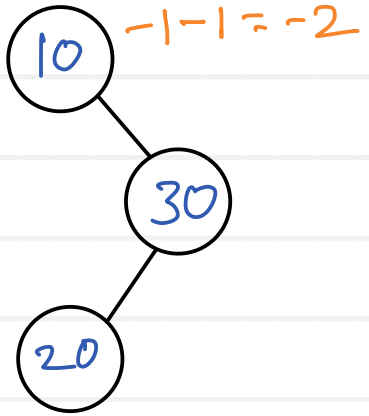
LL Imbalance

Keys : 30, 20, 10



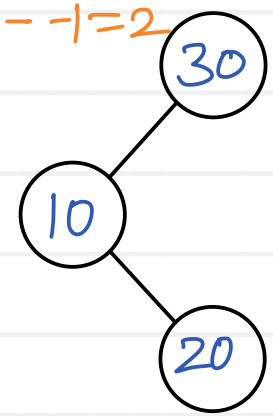
RL Imbalance

Keys : 10, 30, 20



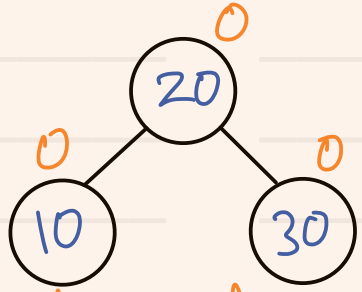
LR Imbalance

Keys : 30, 10, 20



Keys : 20, 10, 30

Keys : 20, 30, 10



Balanced BST

Rotation cases

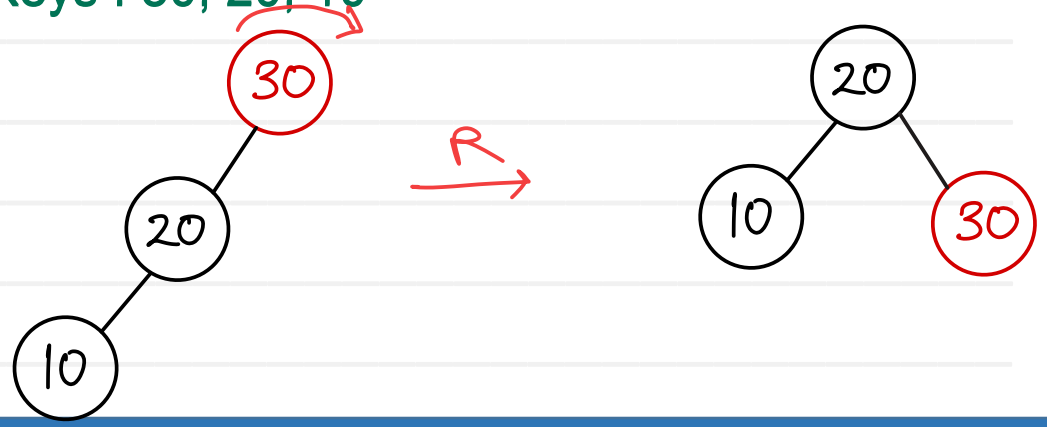
RR Imbalance : apply left rotation on imbalance node

Keys : 10, 20, 30



LL Imbalance : apply right rotation on imbalance node

Keys : 30, 20, 10

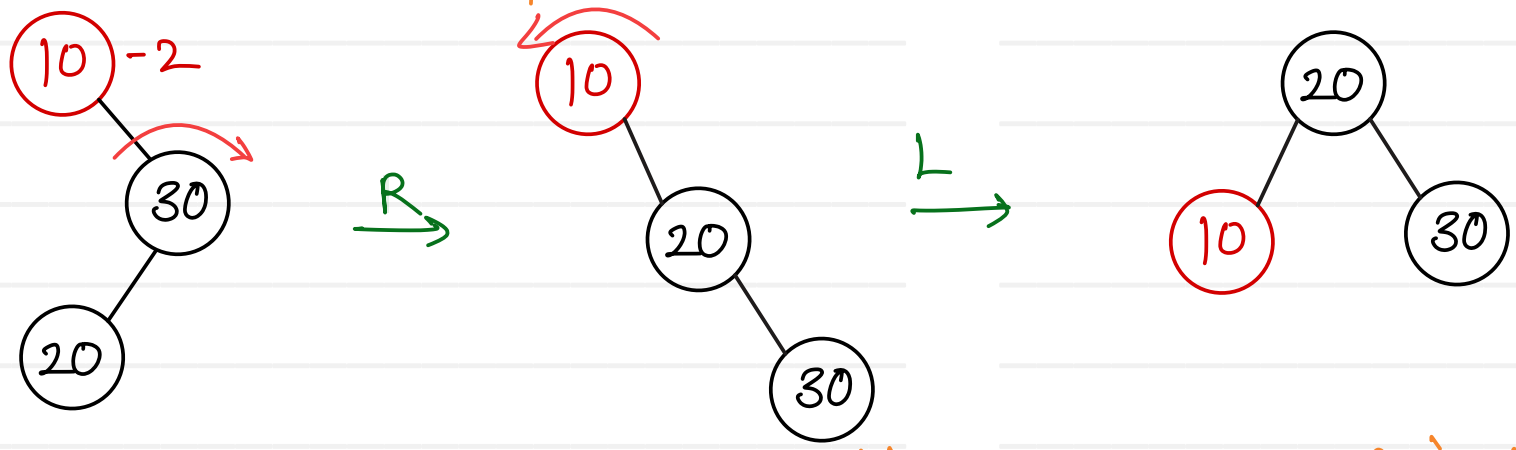


Rotation cases

RL Imbalance

Keys : 10, 30, 20

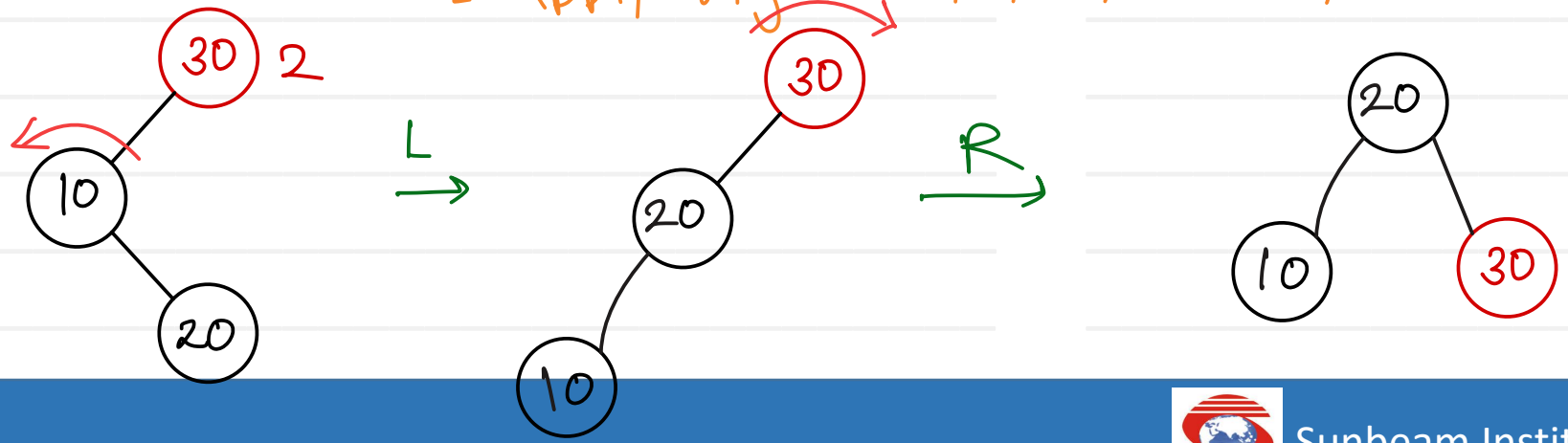
1. apply right rotation on right of imbalance node
2. apply left rotation on imbalance node



LR Imbalance

Keys : 30, 10, 20

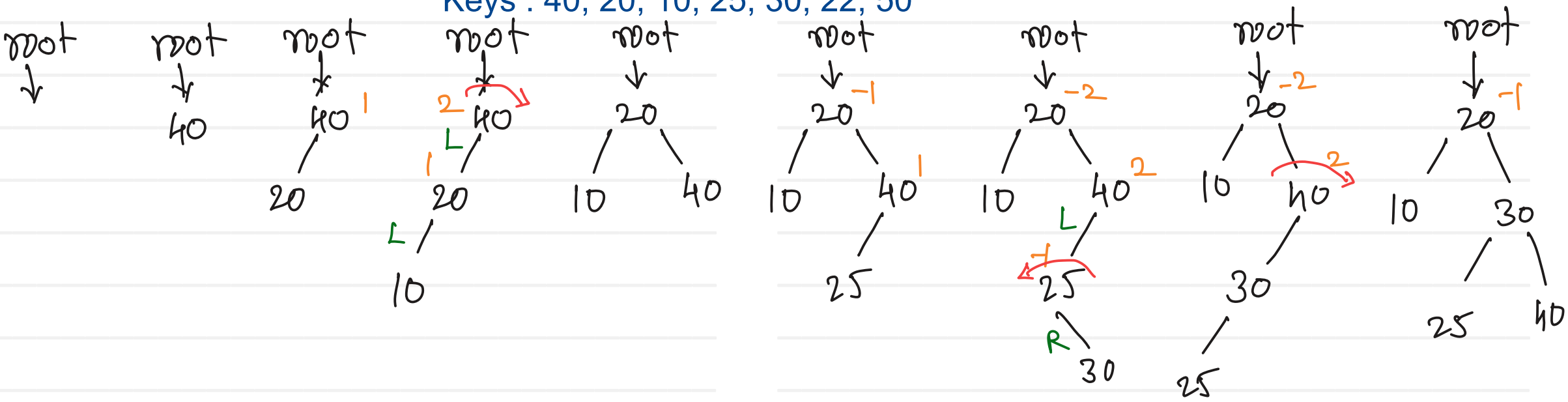
1. apply left rotation on left of imbalance node
2. apply right rotation on imbalance node



AVL Tree

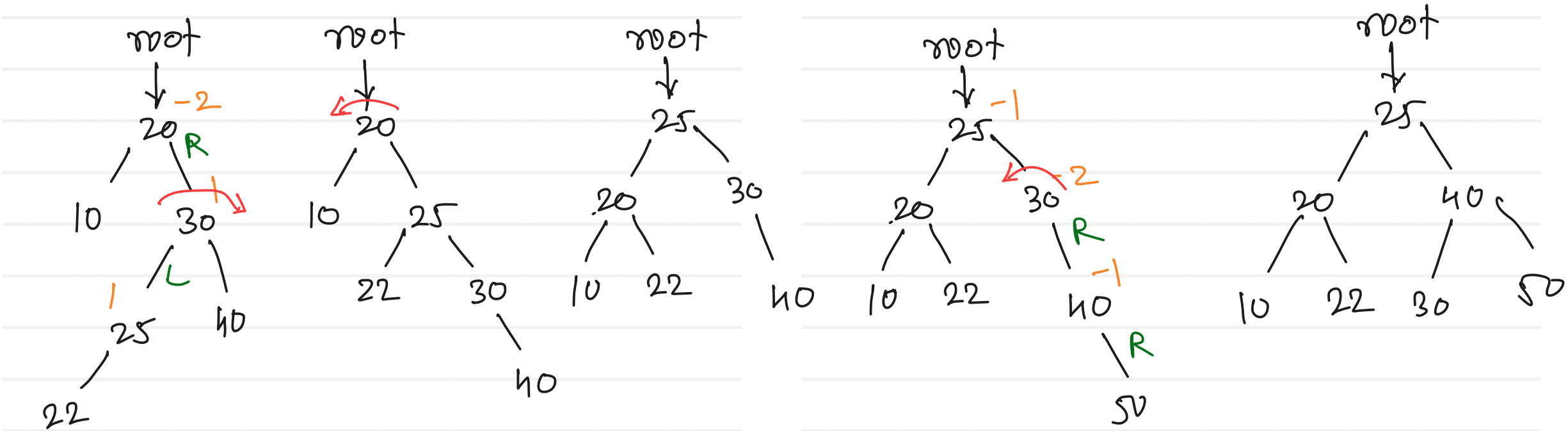
- self balancing binary search tree
- on every insertion and deletion of a node, tree is getting balanced by applying rotations on imbalance nodes
- The difference between heights of left and right sub trees can not be more than one for all nodes
- Balance factors of all the nodes are either -1 , 0 or +1
- All operations of AVL tree are performed in O(log n) time complexity

Keys : 40, 20, 10, 25, 30, 22, 50



AVL Tree

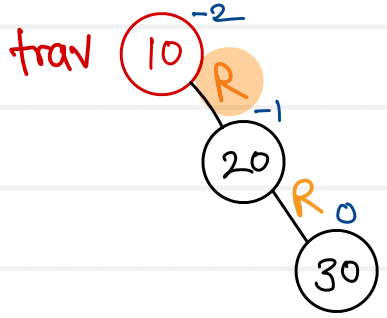
Keys : 40, 20, 10, 25, 30, 22, 50



BF < -1

RR Imbalance

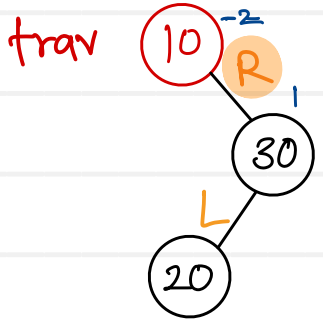
Keys : 10, 20, 30



value > trav.right.data

RL Imbalance

Keys : 10, 30, 20

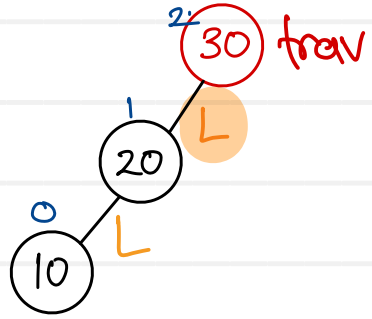


value < trav.right.data

BF > 1

LL Imbalance

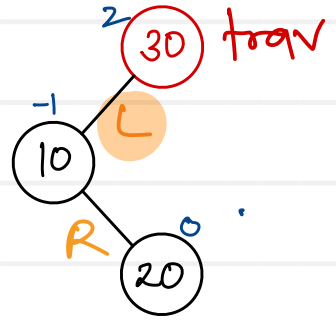
Keys : 30, 20, 10



value < trav.left.data

LR Imbalance

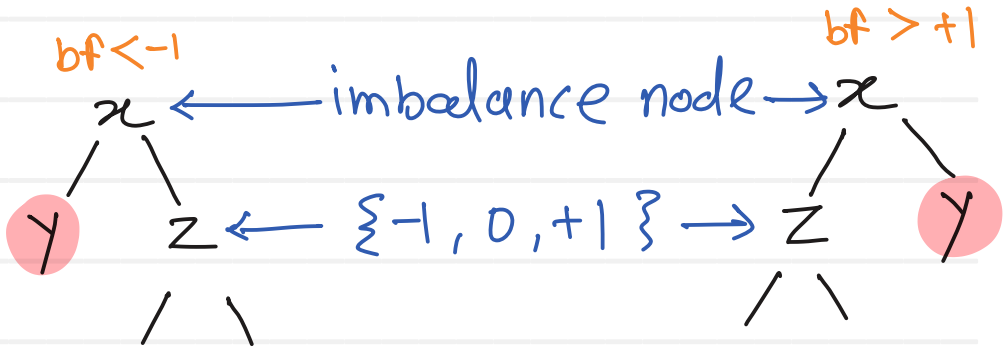
Keys : 30, 10, 20



value > trav.left.data

L-Deletion

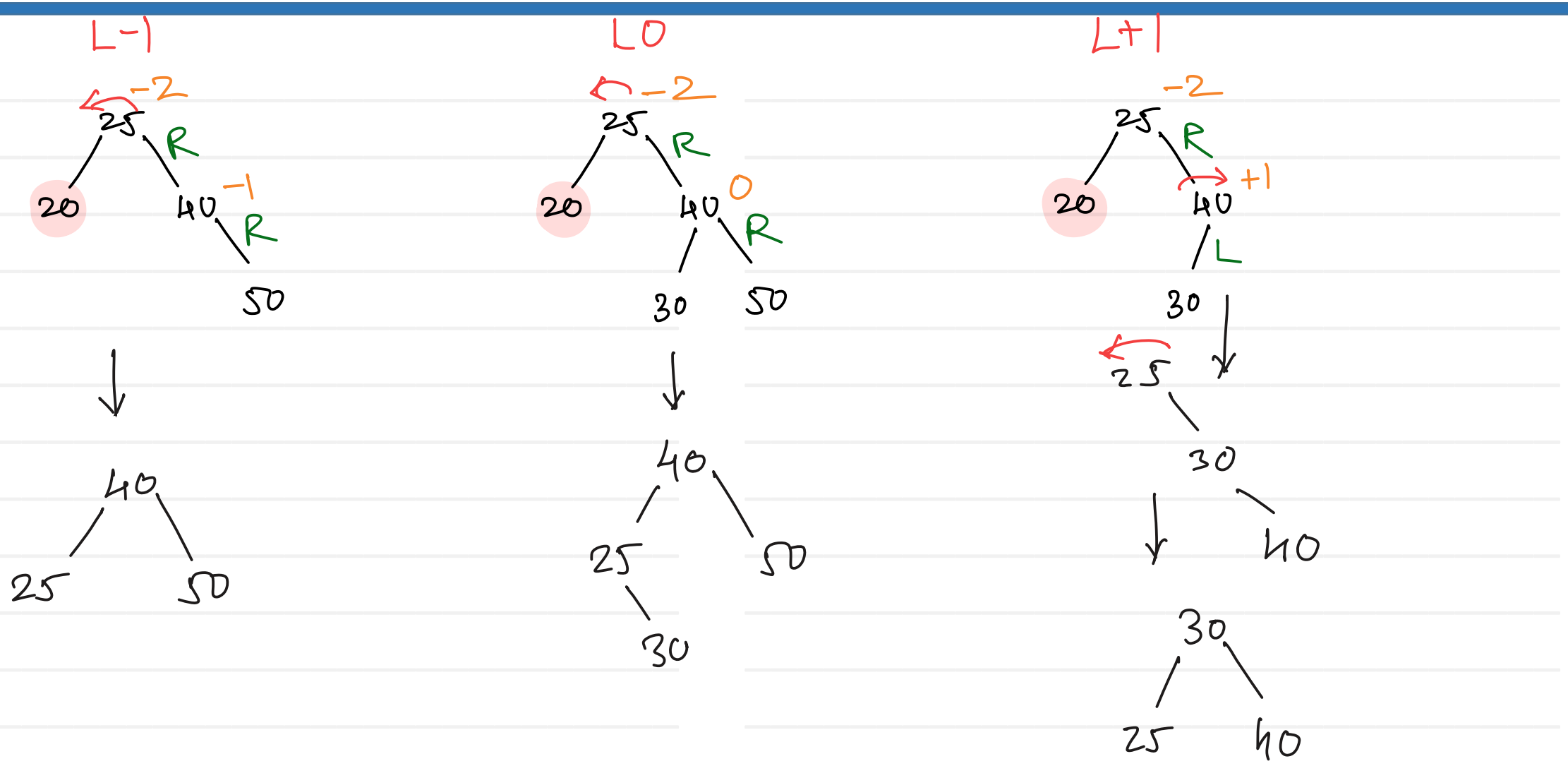
R-Deletion



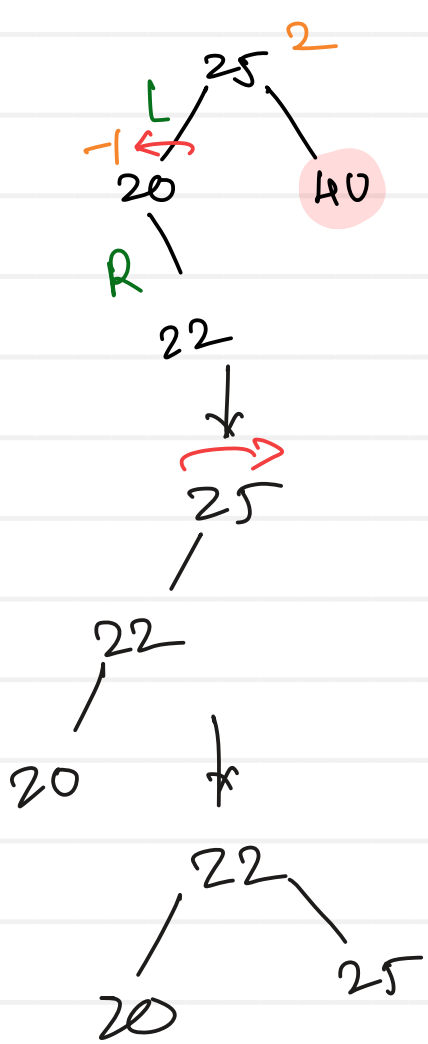
L-1 , L0, L+1

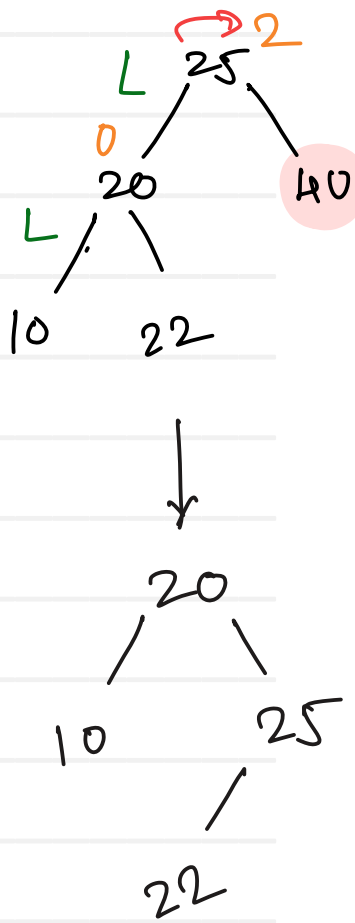
R-1 , R0 , R+1

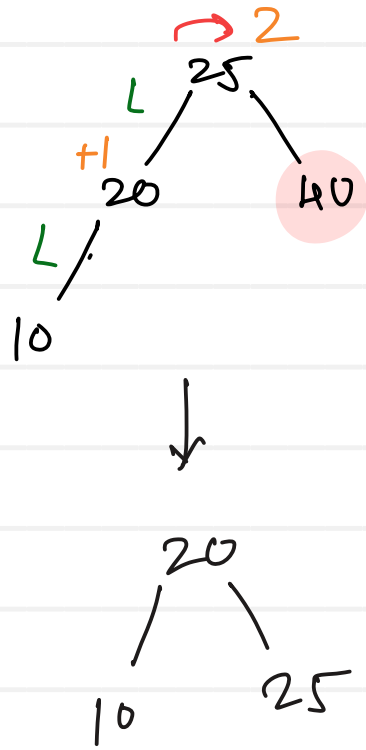
AVL Tree


L-1
-2
25
R
20
40
-1
R
50
40
25
50
L0
-2
25
R
20
40
0
R
30
50
40
25
50
30
L+1
-2
25
R
20
40
+1
L
30
25
30
40
30
25
40

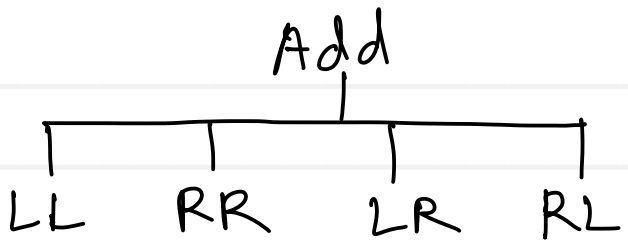
AVL Tree

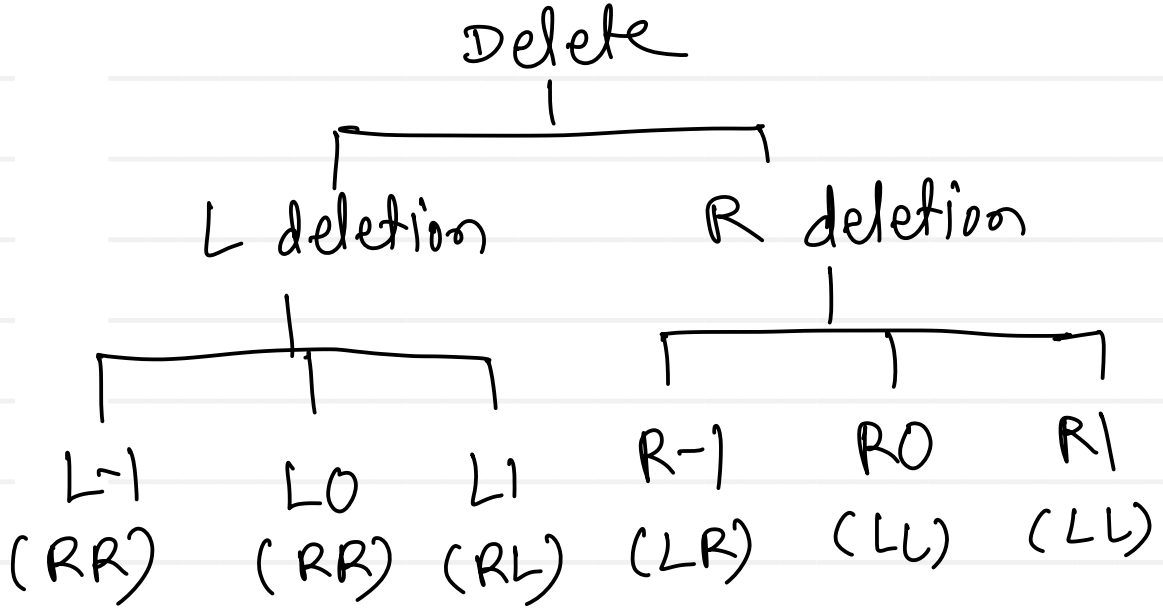

25
2
L
-1
20
40
R
22
25
22
20
22
20
25


2
25
L
0
20
40
L
10
22
20
10
25
22


2
25
L
+1
20
40
L
10
20
10
25

AVL Tree


Add
LL
RR
LR
RL


Delete
L deletion
R deletion
L-1
(RR)
L0
(RR)
L1
(RL)
R-1
(LR)
R0
(LL)
R1
(LL)